

Serverless Quiz Application with AWS DynamoDB and Lambda

A Course Project Report Submitted in partial fulfillment of the course requirements for the award of grades in the subject of

TECHNICAL SKILLING - 4 (CLOUD BASED DATA ANALYTICS SPECIALITY)

by

CHINTA ESHWAR

2110030414

Under the esteemed guidance of

Mrs. P. Sree Lakshmi

Assistant Professor

Department of Computer Science and Engineering



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

K L Deemed to be UNIVERSITY

*Aziznagar, Moinabad , Hyderabad ,
Telangana , Pincode: 500075*

April 2024

K L Deemed to be UNIVERSITY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



Certificate

This is Certified that the project entitled “**Serverless Quiz Application with AWS DynamoDB and Lambda**” which is a **experimental &/ theoretical &/ Simulation&/ hardware** work carried out by **CHINTA ESHWAR (2110030414)**, in partial fulfillment of the course requirements for the award of grades in the subject of **TECHNICAL SKILLING – 4 (CLOUD BASED DATA ANALYTICS SPECIALITY)**, during the year **2023-2024**. The project has been approved as it satisfies the academic requirements.

Mr. Vijay Kumar Damera

Course Coordinator

Dr. Arpita Gupta

Head of the Department

Mrs. P. Sree Lakshmi

Course Instructor

CONTENTS

	Page No.
1. Introduction	1
2. AWS Services Used as part of the project	3
3. Steps involved in solving project problem statement	5
4. Stepwise Screenshots with brief description	7
5. Learning Outcomes	17
6. Conclusion	19
7. References	20

1. INTRODUCTION

In the era of digital learning and engagement, interactive quiz applications have become a popular tool for education, training, and entertainment. Building such applications traditionally required substantial infrastructure setup, but with the advent of serverless computing, developers can now focus more on application logic rather than managing servers. In this guide, we will explore how to create a serverless quiz application using AWS DynamoDB and Lambda, two powerful services offered by Amazon Web Services (AWS).

AWS DynamoDB is a fully managed NoSQL database service that provides fast and predictable performance with seamless scalability. It's an ideal choice for storing structured data like quiz questions, user responses, and other metadata.

AWS Lambda, on the other hand, is a serverless compute service that allows you to run code without provisioning or managing servers. You can use Lambda to execute code in response to triggers such as HTTP requests, database events, or other AWS services. In our case, we'll use Lambda to handle the business logic of our quiz application, such as retrieving questions, scoring responses, and providing feedback to users.

a serverless quiz application using AWS DynamoDB and Lambda, harnessing the scalability and flexibility offered by these AWS services. DynamoDB, a fully managed NoSQL database, will serve as the backbone for storing quiz data, including questions, answers, and user responses. Lambda, AWS's serverless compute service, will handle the application's backend logic, allowing us to focus on functionality without worrying about server management or scalability concerns.

Our goal is to build a robust quiz application that can be used for education, training, or entertainment purposes. Throughout this tutorial, we'll cover everything from designing data models and setting up AWS services to implementing backend logic, building APIs with API Gateway, and developing a simple web frontend.

A serverless quiz application using AWS DynamoDB and Lambda, leveraging their scalability and ease of use. DynamoDB, a NoSQL database, will store quiz data, while Lambda functions will handle application logic without server management overhead. Our steps include designing data models, setting up AWS services, implementing backend logic, building APIs with API Gateway, and developing a simple web frontend.

2. AWS SERVICES USED AS PART OF THE PROJECT

AWS DYNAMODB:

AWS DynamoDB is a fully managed NoSQL database service provided by Amazon Web Services. It offers seamless scalability, high availability, and single-digit millisecond latency for read and write operations. DynamoDB supports flexible schemas, automatic scaling, and integrates with various AWS services. It's designed for real-time applications, ensuring consistent performance and durability. DynamoDB replicates data across multiple Availability Zones for fault tolerance and offers robust security features including encryption at rest and in transit. It integrates with AWS IAM for access control and AWS KMS for encryption key management. DynamoDB is commonly used for web and mobile applications, gaming, IoT, and real-time analytics. It's accessible via AWS SDKs, enabling developers to build scalable and serverless applications easily.

AWS LAMBDA:

AWS Lambda is a serverless compute service by Amazon Web Services. It allows developers to run code without provisioning or managing servers. Lambda functions are triggered by events from various sources, scaling automatically to handle incoming requests. With support for multiple programming languages, Lambda seamlessly integrates with other AWS services, offering flexible configuration options. It charges users based on the compute time consumed by their functions, making it cost-effective for event-driven applications and microservices architectures.

AWS API GATEWAY:

AWS API Gateway is a fully managed service by Amazon Web Services, facilitating the creation, publishing, and management of APIs at any scale. It acts as a front door for applications to access data and services hosted on AWS or elsewhere. API Gateway supports various API types and integrates seamlessly with other AWS services like Lambda, DynamoDB, and S3. It offers features for request and response transformations, security and authentication, traffic management, monitoring, and analytics. Developers can implement fine-grained access control, manage API usage quotas and rate limits, and monitor API performance. API Gateway supports custom domain names and SSL certificates for secure communication. Overall, it simplifies the

process of building and managing APIs, enabling developers to expose backend services securely and efficiently.

IAM:

AWS IAM (Identity and Access Management) is a service provided by Amazon Web Services that enables users to securely control access to AWS services and resources. IAM allows users to manage permissions for users, groups, and roles within their AWS account. It provides fine-grained access control, allowing administrators to grant or deny permissions to individual resources based on policies. IAM integrates with other AWS services, enabling users to control who can access specific services and perform actions within those services. IAM also supports multi-factor authentication (MFA) for added security and compliance. It offers features for managing user credentials, such as access keys and passwords, and supports encryption of data at rest and in transit. IAM is essential for securing AWS resources and ensuring that only authorized users have access to sensitive data and services. Overall, it plays a critical role in maintaining the security and integrity of AWS environments.

3.STEPS INVOLVED IN SOLVING PROJECT PROBLEM STATEMENT

Steps involved in solving the problem statement of building a Serverless Quiz Application with AWS DynamoDB and Lambda:

1.Requirement Analysis:

Understand the project's objectives and requirements.

Identify key features such as user authentication, question management, answer submission, scoring mechanism, and result feedback.

2.Design Data Model:

Define the data schema for DynamoDB tables based on the application's requirements. Determine the structure for storing quiz questions, user responses, metadata, and any other relevant information.

Establish primary keys, sort keys, and attributes for efficient data retrieval and storage.

3.Set Up AWS Services:

Create an AWS account if not already done.

Set up DynamoDB tables based on the defined data model.

Configure permissions and roles using AWS IAM to allow Lambda functions to interact with DynamoDB.

Create Lambda functions to handle various aspects of the quiz application, such as fetching questions, validating responses, calculating scores.

4.Implement Backend Logic:

Write the business logic for Lambda functions using supported programming languages (e.g., Python, Node.js).

Implement functions to interact with DynamoDB tables, such as querying questions, validating answers, and updating user responses.

Ensure error handling and logging mechanisms are in place to handle exceptions gracefully and provide debugging information.

5.Configure API Gateway:

Set up RESTful APIs using AWS API Gateway to expose Lambda functions to the frontend.

Define API endpoints for actions like fetching quiz questions, submitting answers, and retrieving results.

6.Develop Frontend:

Design a user-friendly web interface using HTML, CSS, and JavaScript to interact with the backend API.

Create UI components for displaying quiz questions, capturing user responses, and presenting results.

Implement client-side logic to handle user interactions and communicate with the backend API endpoints via AJAX or fetch API.

7.Testing:

Conduct unit tests for Lambda functions to ensure they perform as expected.

Perform integration tests to validate the interaction between frontend components and backend APIs.

Test the application's functionality across different browsers and devices to ensure compatibility and responsiveness.

Execute end-to-end tests to verify the flow of actions from user input to backend processing and result presentation.

8.Deployment:

Deploy the serverless quiz application to AWS using deployment tools like AWS SAM (Serverless Application Model) or AWS CLI.

Configure necessary resources such as DNS settings, SSL certificates, and custom domain names if applicable.

Verify that the application is accessible and functional in the production environment.

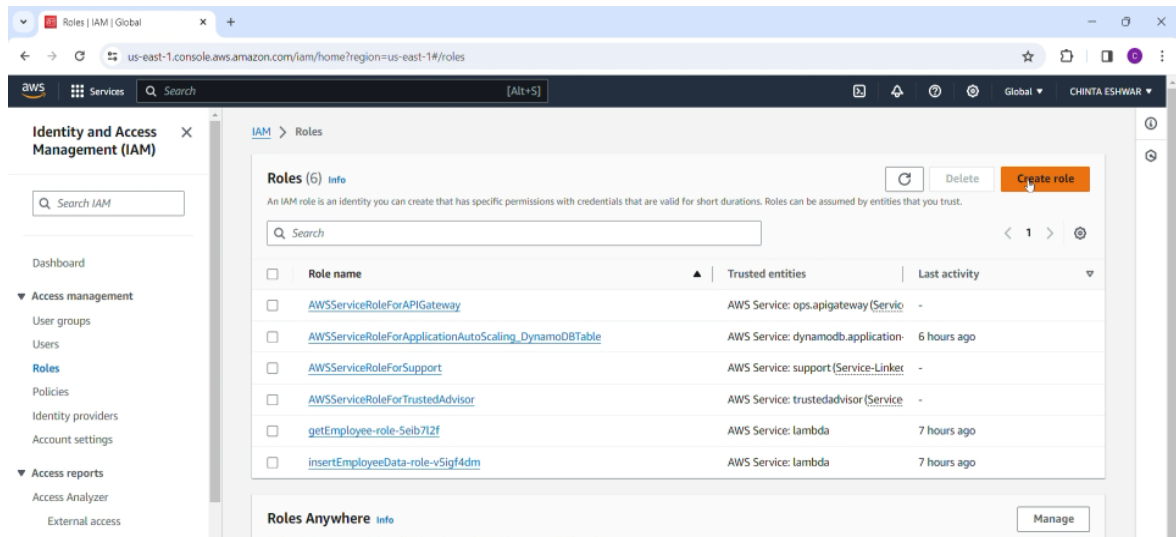
9.Monitoring and Maintenance:

Monitor DynamoDB and Lambda resource usage to optimize performance and manage costs effectively.

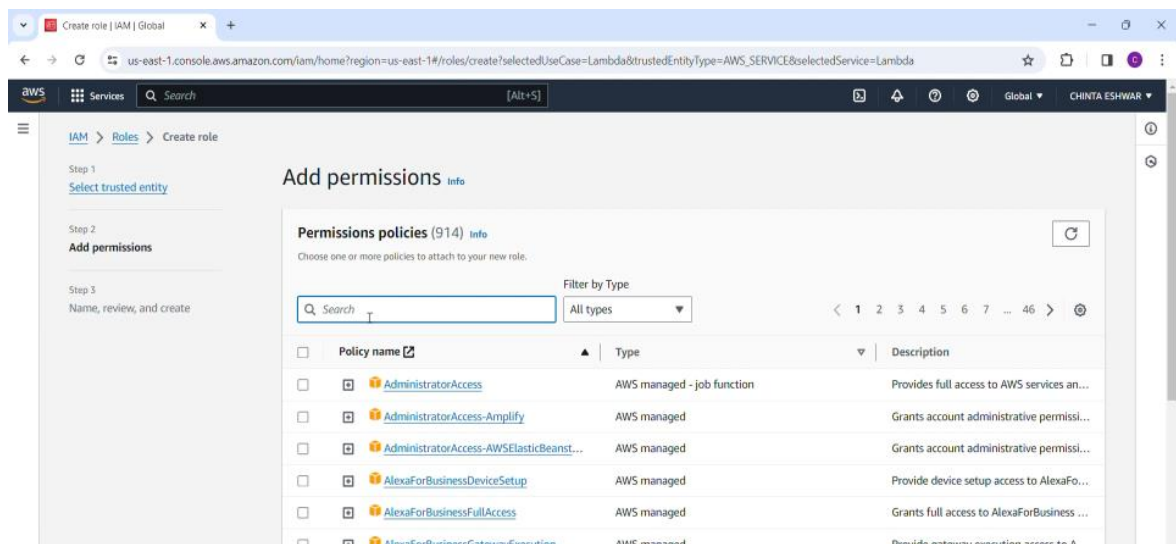
4. STEPWISE SCREENSHOTS WITH BRIEF DESCRIPTION

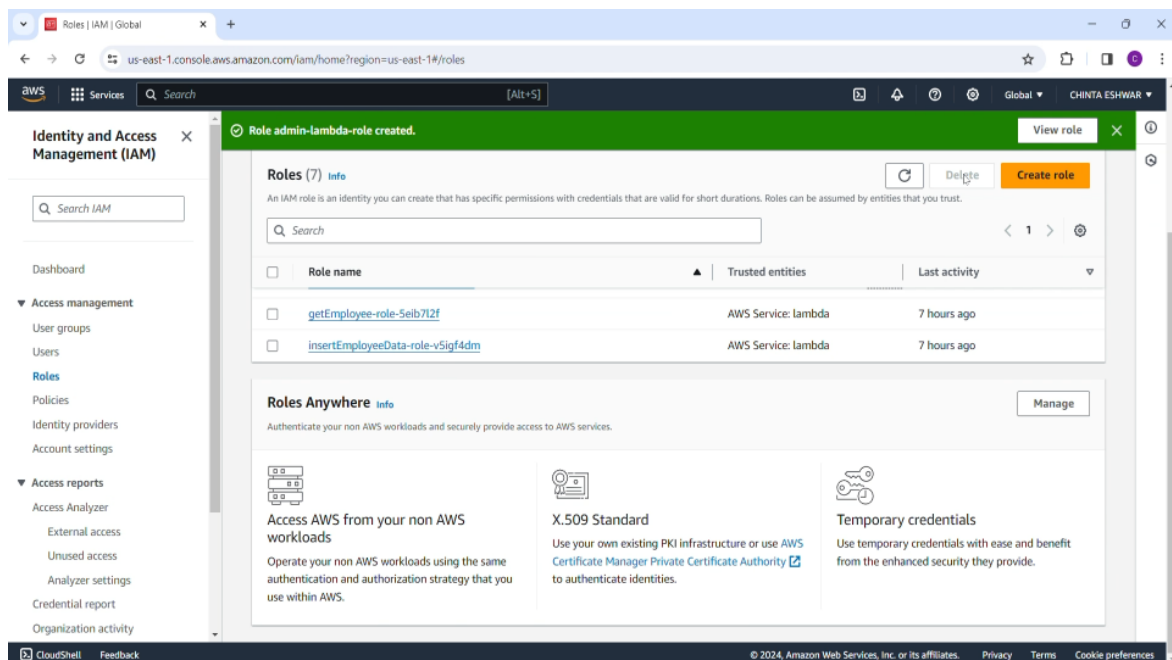
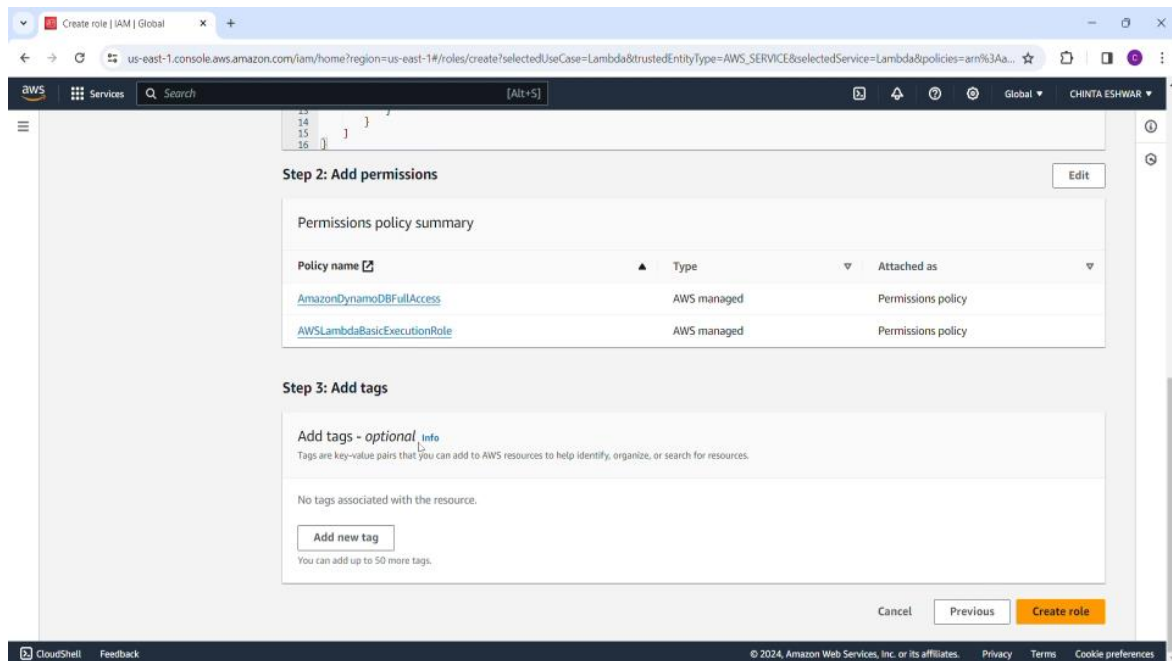
CREATING IAM ROLE AND ADDING PERMISSIONS:

Creating an IAM role is an essential step in granting permissions to AWS resources like Lambda functions to access other AWS services like DynamoDB.



Add AwsLambdaBasicExecutionRole and AmazonDynamoDBFullAccess permissions





CREATING LAMBDA FUNCTION:

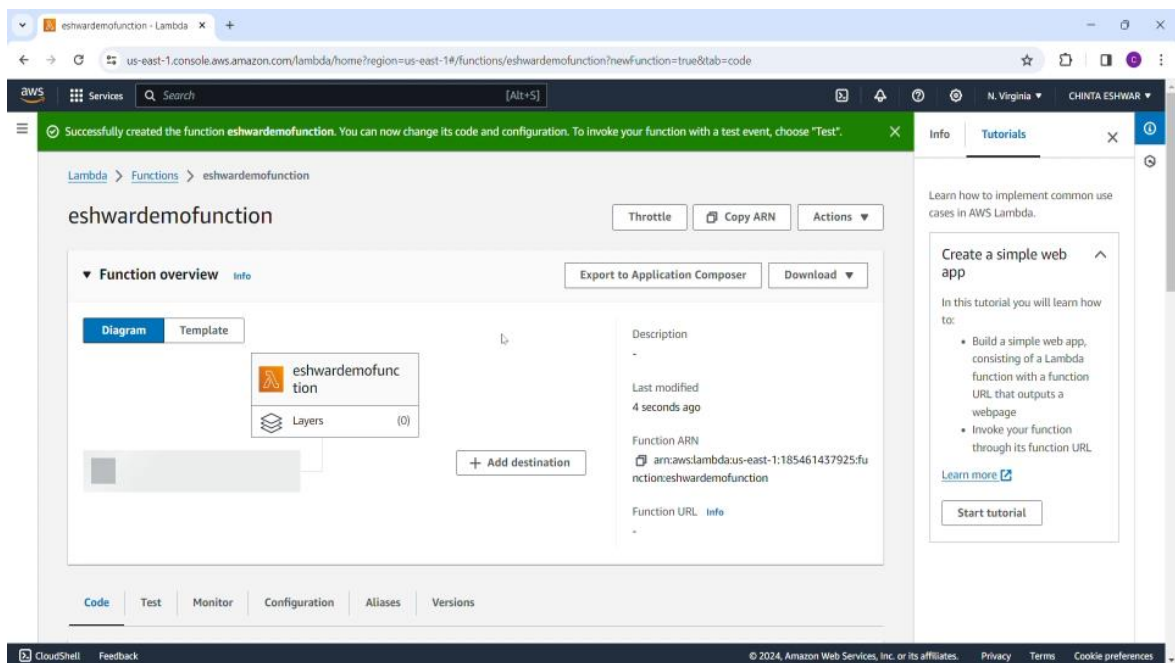
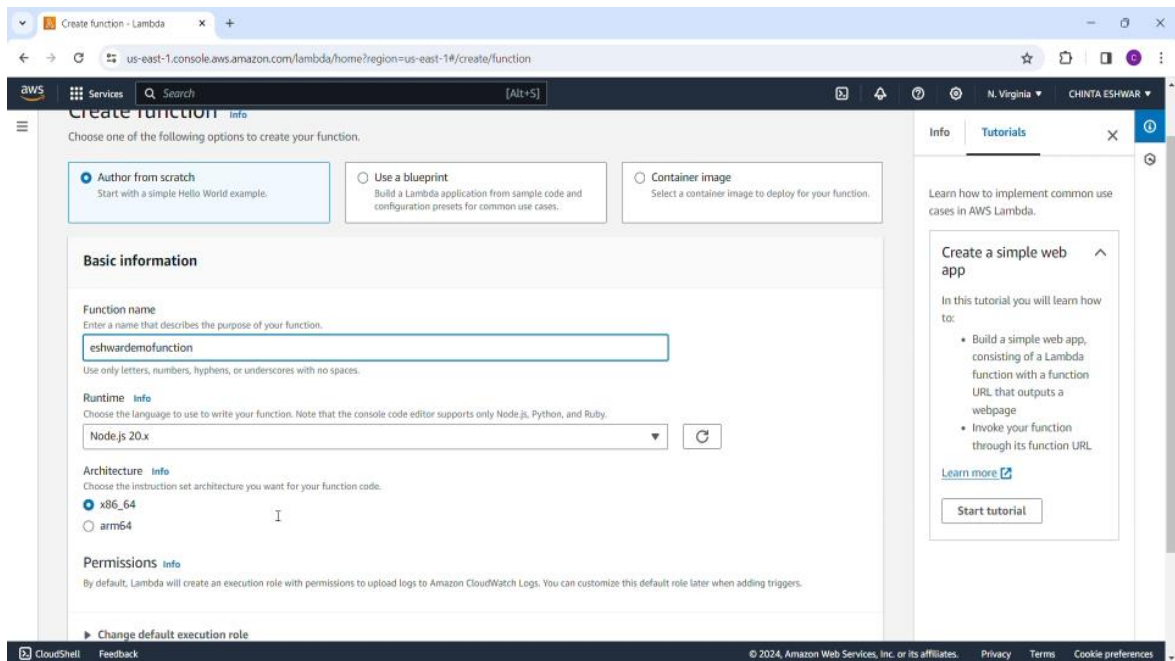
In the Lambda dashboard, click on the "Create function" button to start creating a new Lambda function.

Enter a name for your Lambda function.

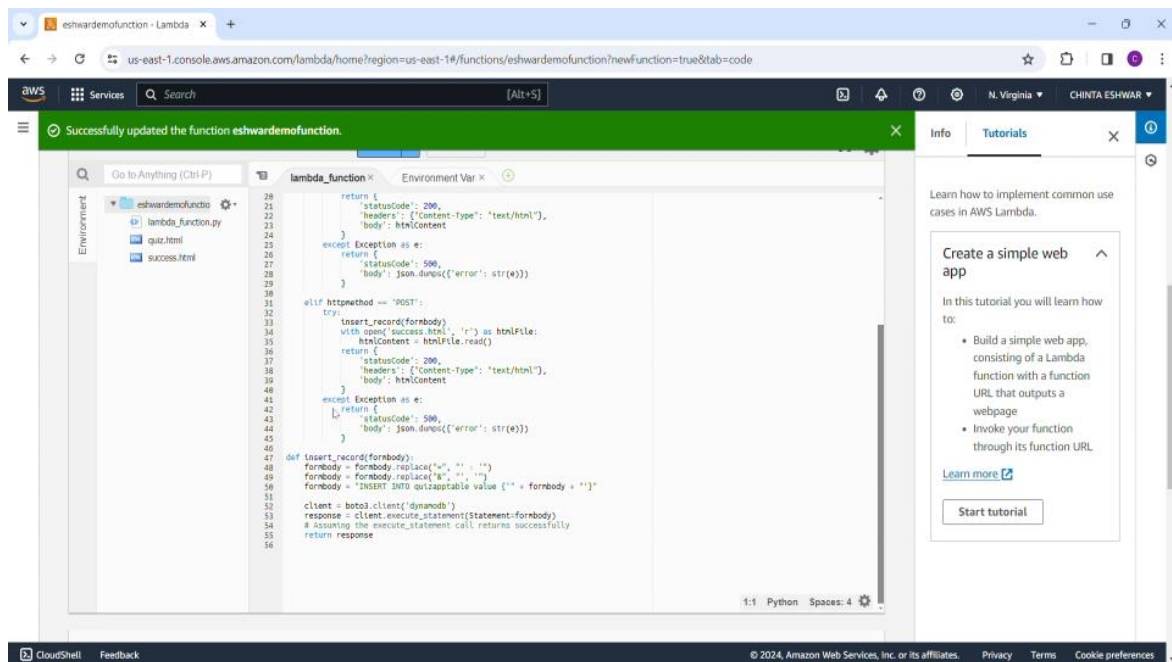
Choose a runtime environment, such as Node.js, Python, Java, etc.

Select an appropriate execution role. If you don't have an existing role, you can create a new one or use an existing role with necessary permissions.

Once you've configured the basic information, click on the "Create function" .



Add the zip file



CREATE DYNAMODB TABLE:

In the DynamoDB dashboard, click on the "Create table" button to start creating a new DynamoDB table.

Enter a name for your DynamoDB table in the "Table name" field.

Define the primary key for your table. This consists of a partition key and an optional sort key. These keys uniquely identify each item in the table.

Choose the data types for the partition key and sort key, if applicable.

Once you've configured the table details, capacity settings, and any additional indexes or advanced settings, click on the "Create" button to create your DynamoDB table.

Once the table creation is complete, you'll see a confirmation message indicating that the table has been created successfully.

Your DynamoDB table is now ready to use, and you can start adding data to it or querying it using the AWS SDK or DynamoDB API.

us-east-1.console.aws.amazon.com/dynamodbv2/home?region=us-east-1#create-table

Create table

Table details Info

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name

This will be used to identify your table.

quizapptable

Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).

Partition key

The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.

Enter the partition key name String

1 to 255 characters and case sensitive.

Sort key - optional

You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.

Enter the sort key name String

1 to 255 characters and case sensitive.

Table settings

CloudShell Feedback

© 2024, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

us-east-1.console.aws.amazon.com/dynamodbv2/home?region=us-east-1#create-table

Capacity mode Provisioned Yes

Provisioned read capacity 5 RCU Yes

Provisioned write capacity 5 WCU Yes

Auto scaling On Yes

Local secondary indexes - No

Global secondary indexes - Yes

Encryption key management Owned by Amazon DynamoDB Yes

Deletion protection Off Yes

Tags

Tags are pairs of keys and optional values, that you can assign to AWS resources. You can use tags to control access to your resources or track your AWS spending.

No tags are associated with the resource.

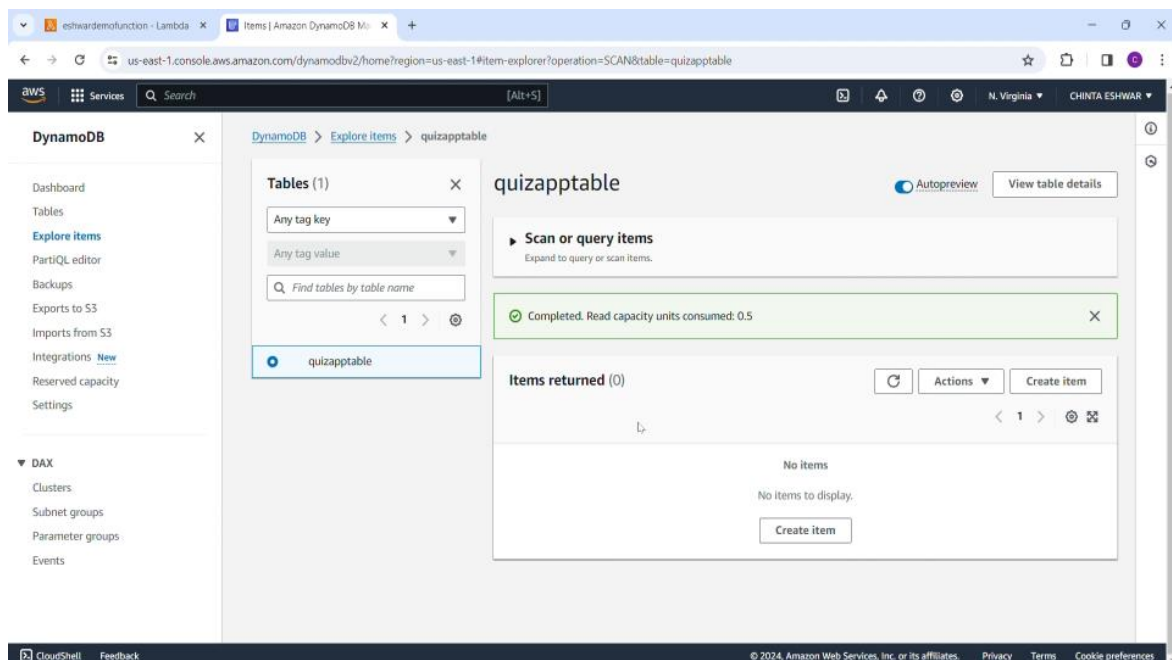
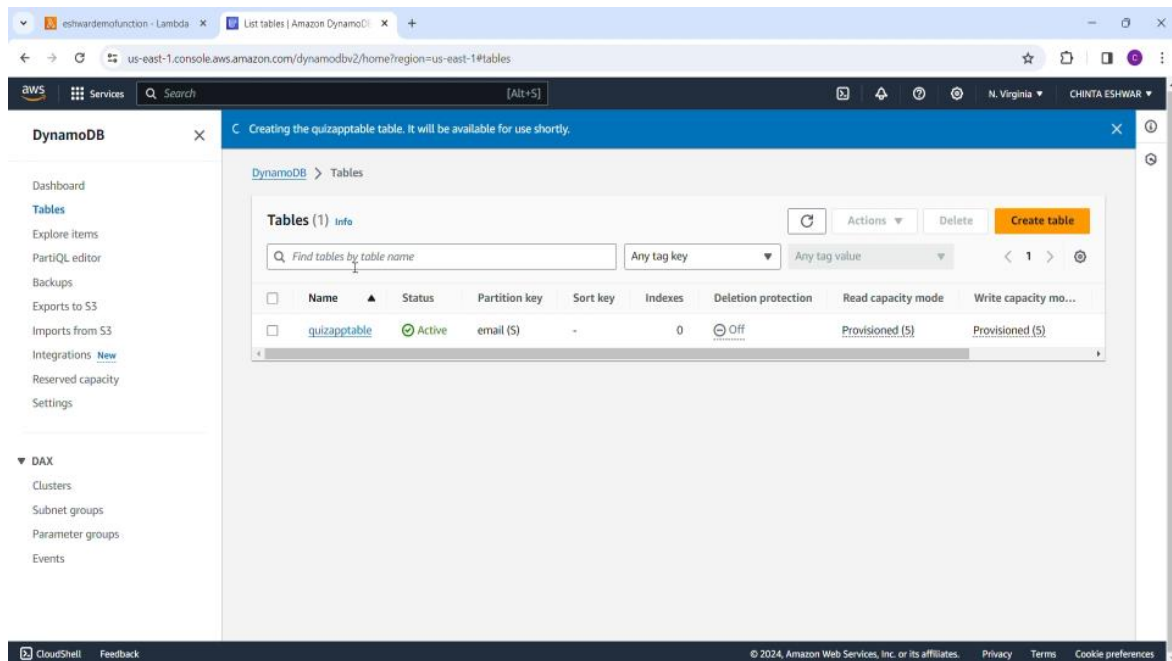
Add new tag

You can add 50 more tags.

Cancel Create table

CloudShell Feedback

© 2024, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences



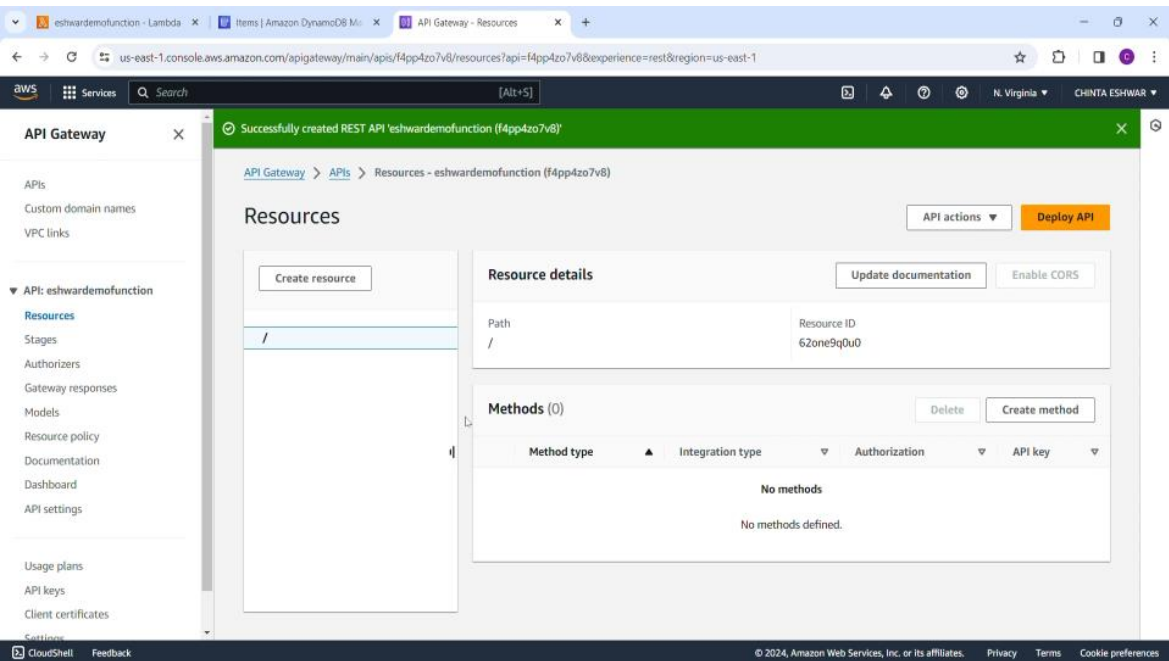
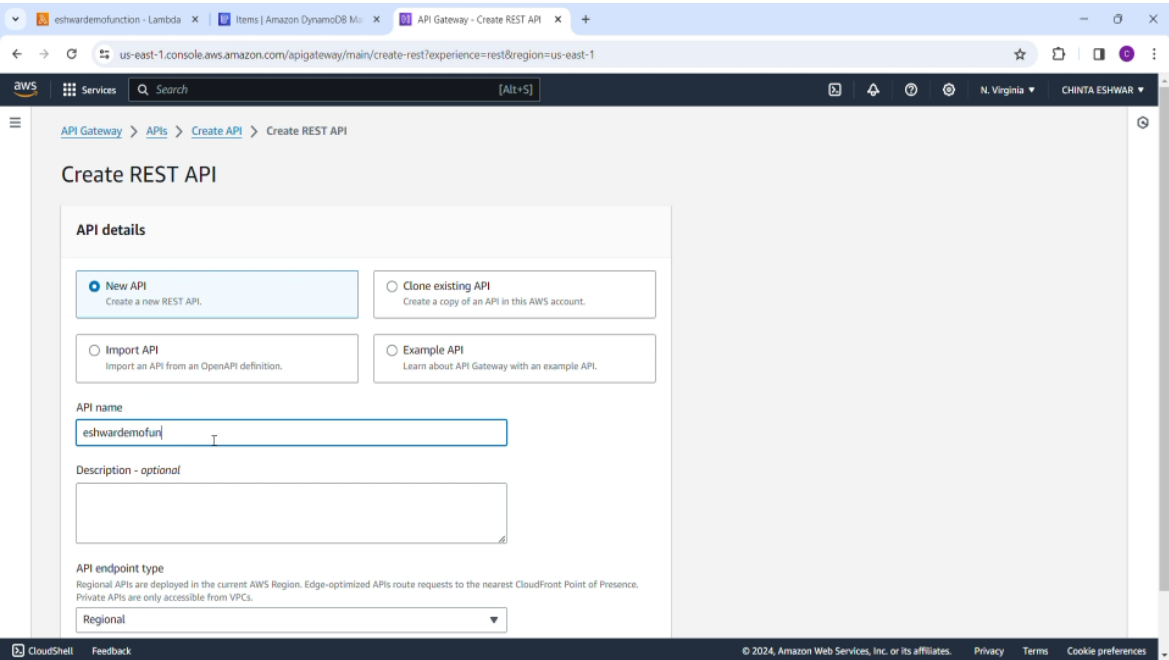
CREATING REST API:

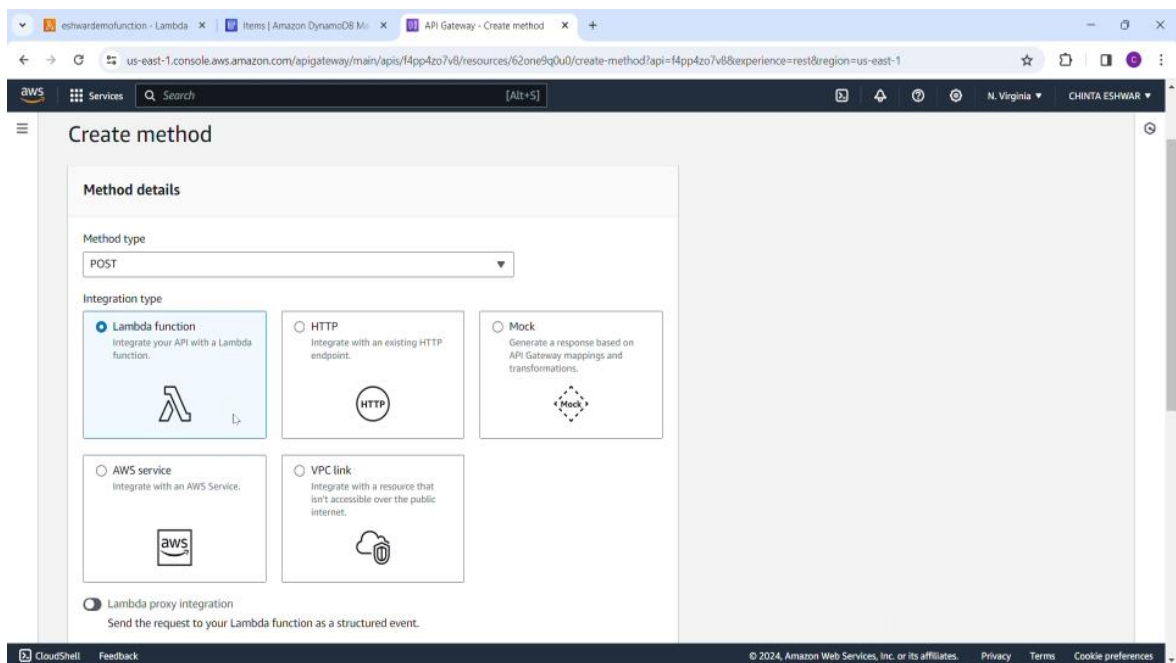
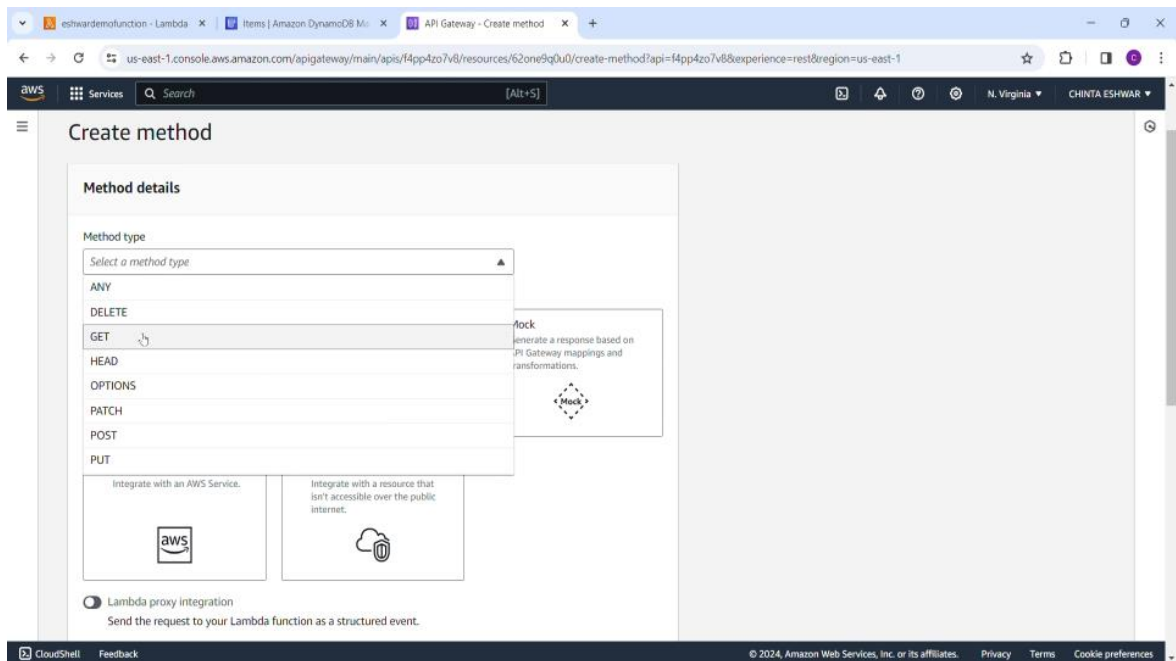
In the API Gateway dashboard, click on the "Create API" button to start creating a new API.
Create Rest API.

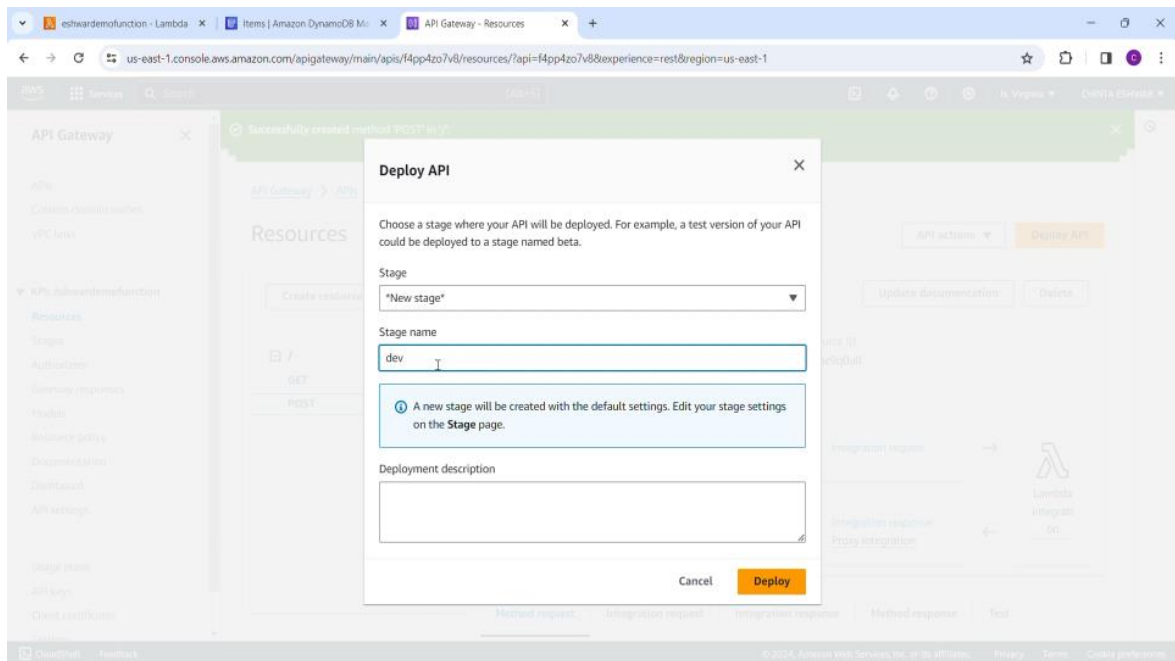
Once you've created a resource, click on it to define the methods supported by that resource.
Click on the "Actions" dropdown menu and select "Create Method".

Choose the HTTP method get method and post method.

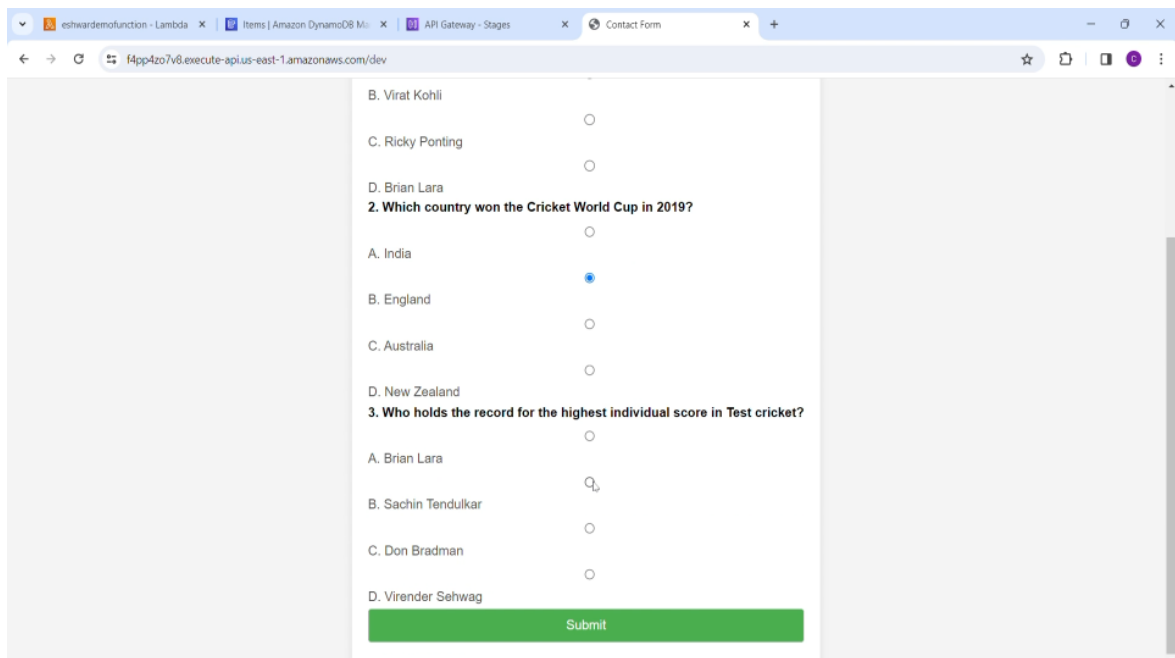
Once you've configured your API resources and methods, click on the "Actions" dropdown menu and select "Deploy API".
Choose a stage to deploy your API.



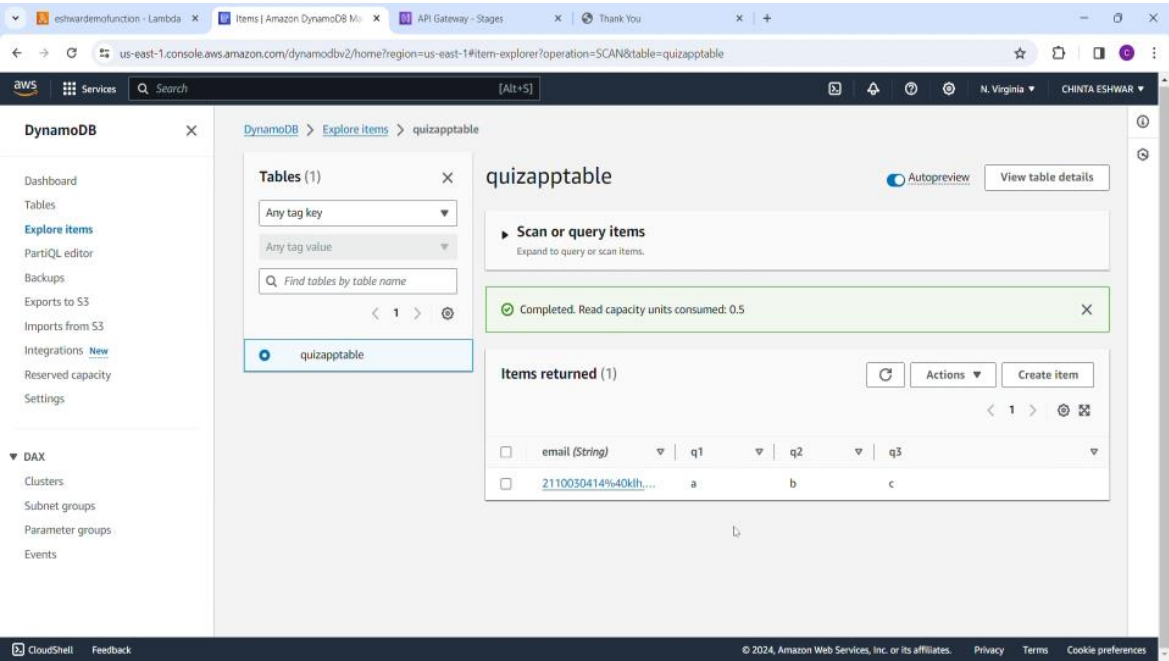




COPY THE URL AND PASTE IN NEW BROWSER TAB:



WE CAN SEE TABLE IN DYNAMODB:



5. LEARNING OUTCOMES

Building a Serverless Quiz Application with AWS DynamoDB and Lambda provides valuable learning outcomes, including:

Understanding of Serverless Architecture: Gain a deep understanding of serverless architecture principles and how to leverage managed services like AWS Lambda and DynamoDB to build scalable and cost-effective applications without managing infrastructure.

Hands-on Experience with AWS Services: Acquire practical experience in using AWS services such as DynamoDB, Lambda, API Gateway, IAM gaining proficiency in configuring and integrating these services to create a complete application.

Database Design and Data Modeling: Learn database design principles and data modeling techniques specific to NoSQL databases like DynamoDB. Understand how to design efficient schemas and utilize features like primary keys and secondary indexes.

Backend Development Skills: Develop backend development skills by writing business logic in serverless functions using programming languages like Python or Node.js. Learn how to handle data retrieval, validation, and processing within Lambda functions.

API Design and Integration: Gain expertise in designing RESTful APIs using API Gateway and integrating them with Lambda functions.

Testing and Debugging: Learn testing strategies for serverless applications, including unit testing Lambda functions, integration testing API endpoints.

Scalability and Performance Optimization: Learn how to design serverless applications for scalability and optimize performance by leveraging features like auto-scaling, caching, and asynchronous processing.

Security and Compliance: Gain knowledge of security best practices for serverless applications, including IAM roles and policies, encryption at rest and in transit, and compliance requirements for handling sensitive data.

Project Management and Documentation: Develop project management skills by planning, executing, and documenting the development process of a real-world application, including requirements analysis, design decisions, implementation details, and user documentation.

6. CONCLUSION

In conclusion, building a Serverless Quiz Application with AWS DynamoDB and Lambda offers a rich learning experience and a practical demonstration of serverless architecture principles and learned how to write serverless functions to handle business logic, interact with DynamoDB tables, and expose functionality via RESTful APIs using API Gateway and gained valuable insights into designing, implementing, and deploying serverless applications using managed services provided by Amazon Web Services.

7. REFERENCES

1. Github: <https://github.com/avizway1/aws-projects/blob/main/Project-4%20Lambda%20API%20Gateway%20and%20DynamoDB/projectfunction.zip>
2. Youtube: <https://www.youtube.com/watch?v=dsH2QC6O3Gg>
3. Tutorial: Build and Test a Serverless Application with AWS Lambda:
<https://docs.aws.amazon.com/toolkit-for-visual-studio/latest/user-guide/lambda-build-test-serverless-app.html>
4. <https://blog.ordina-jworks.io/cloud/2018/10/01/How-to-build-a-Serverless-Application-with-AWS-Lambda-and-DynamoDB.html>
5. <https://www.youtube.com/watch?v=VGerk8hrP9U>